

Memoria Técnica

OCR Layout + Multi-OCR + Evaluación con Ground Truth

Ángel Manuel Pereira Rodríguez

9 de marzo de 2026

Resumen

Esta memoria técnica describe el desarrollo completo del proyecto de visión artificial orientado a documentos históricos escaneados. El sistema combina varios métodos de detección de layout, varios motores OCR y un proceso de evaluación con ground truth para medir calidad real de extracción. A lo largo del proyecto se implementó un pipeline reproducible que cubre detección de regiones de texto, búsqueda sistemática de configuraciones, ranking heurístico, validación OCR multi-motor y comparación final mediante CER, WER y métricas de orden de lectura. La memoria resume la arquitectura, las imágenes utilizadas, los resultados obtenidos y el procedimiento de instalación y uso.

Índice

1. Introducción	3
2. Objetivos del proyecto	3
3. Arquitectura general del sistema	4
4. Imágenes utilizadas	4
5. Proceso de instalación del sistema	7
5.1. Requisitos previos	7
5.2. Instalación recomendada	7
5.3. Notas de compatibilidad importantes	8
6. Proceso de uso del sistema	8
6.1. Detección de layout	8
6.2. OCR por imagen o por carpeta	8
6.3. Experimentación y ranking de layout	8
6.4. Validación OCR multi-motor	9
6.5. Comparación contra ground truth	9
7. Desarrollo experimental del proyecto	9
7.1. Fase 1: detección de layout	9
7.2. Fase 2: grid search	9
7.3. Fase 3: ranking heurístico	10
7.4. Fase 4: validación OCR multi-motor	11
7.5. Fase 5 y 6: ground truth y evaluación objetiva	13
8. Discusión de resultados	13

9. Conclusiones

13

Índice de figuras

1.	Galería del dataset, bloque 1 de 3.	5
2.	Galería del dataset, bloque 2 de 3.	6
3.	Galería del dataset, bloque 3 de 3.	6

Índice de cuadros

1.	Resumen del conjunto de imágenes utilizado	5
2.	Inventario del conjunto de imágenes	6
3.	Top 10 global del ranking de configuraciones de layout	10
4.	Campos utilizados en las tablas de ranking de layout	10
5.	Mejor configuración obtenida por cada familia de layout	10
6.	Campos utilizados en la comparativa por familia de layout	11
7.	Comparativa de motores OCR sobre la mejor familia de layout (OpenCV)	11
8.	Mejor resultado encontrado por familia de layout usando OCR DeepSeek	11
9.	Top 20 combinaciones OCR-layout por accuracy de contenido	12
10.	Tabla complementaria: Top 20 combinaciones OCR-layout por velocidad	12
11.	Campos utilizados en las tablas OCR y de comparación con ground truth	12

1 Introducción

El objetivo del proyecto ha sido construir un sistema robusto para analizar documentos impresos antiguos con OCR, teniendo en cuenta que el principal cuello de botella no es solo el motor de reconocimiento, sino también la calidad del layout previo. En este tipo de material aparecen varios problemas combinados:

- Páginas dobles o composiciones con varias columnas,
- Anuncios y zonas decorativas mezcladas con texto,
- Escaneos envejecidos, pliegues, sombras y baja uniformidad,
- Necesidad de mantener un orden de lectura razonable.

Para resolverlo, el proyecto no se limitó a aplicar OCR directamente sobre la imagen completa. En su lugar, se diseñó un pipeline por fases:

1. Detección de layout por varios métodos,
2. Experimentación sistemática de configuraciones,
3. Ranking de configuraciones de detección,
4. Validación OCR con varios motores sobre las mismas regiones,
5. Comparación final frente a ground truth.

Este enfoque permite separar con claridad dos tipos de error:

- Errores de segmentación del documento,
- Errores de reconocimiento del texto.

2 Objetivos del proyecto

Los objetivos principales han sido los siguientes:

- Detectar regiones de texto de forma fiable en documentos históricos escaneados,
- Comparar distintos métodos de layout en un mismo conjunto de imágenes,
- Medir el comportamiento de varios motores OCR bajo condiciones equivalentes,
- Crear un ground truth de referencia para evaluación objetiva,
- Dejar un sistema instalable, reproducible y extensible.

La unidad real de evaluación del sistema es:

`configuración_de_layout x motor_ocr x imagen`

3 Arquitectura general del sistema

El repositorio está organizado alrededor de varios scripts especializados. Los más importantes son:

- `detect_columns.py`: Detección de layout por múltiples métodos.
- `post_processing.py`: NMS, merge y limpieza de cajas.
- `experiment_models.py`: Grid search sobre configuraciones de layout.
- `analyze_experiments.py`: Ranking agregado de configuraciones.
- `validate_ocr.py`: Validación multi-motor OCR.
- `compare_validation_vs_ground_truth.py`: Comparación con ground truth.
- Scripts OCR individuales para pruebas por motor y depuración comparativa.

Los métodos de layout implementados son:

- `opencv`: Enfoque heurístico, muy rápido, sin modelos profundos.
- `doclayout`: DocLayout-YOLO basado en YOLOv10 y DocStructBench.
- `yolo11`: Modelo YOLO11 fine-tuned sobre DocLayNet.
- `paddleocr`: Detector de layout de PaddleOCR PP-StructureV3.
- `docling`: Detector de layout de IBM Docling.

Los motores OCR comparados son:

- EasyOCR,
- Tesseract,
- PaddleOCR,
- DeepSeek-OCR.

4 Imágenes utilizadas

El conjunto de trabajo utilizado en el proyecto está compuesto por 18 imágenes en la carpeta `imgs/`, nombradas de `popurri01.jpg` a `popurri18.jpg`. Todas ellas pertenecen a material histórico escaneado relacionado con agrupaciones, anuncios, popurrís y documentos de carnaval, con gran variedad de diseños y distribuciones.

Este conjunto resulta útil porque mezcla casos relativamente limpios con otros mucho más difíciles, incluyendo:

- Páginas simples con una sola masa textual,
- Dobles páginas con varias columnas,
- Anuncios tipográficos decorativos,
- Documentos con ilustraciones o botellas,

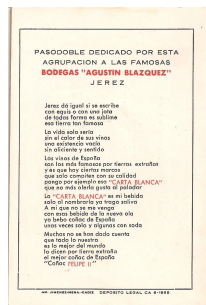
- Fichas biográficas con texto continuo de alto contraste.

Cuadro 1: Resumen del conjunto de imágenes utilizado

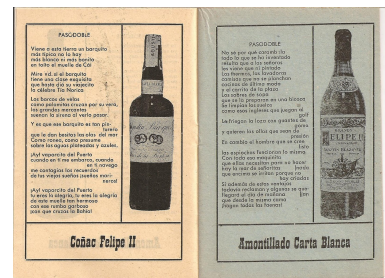
Elemento	Descripción
Conjunto principal	18 imágenes popurri01.jpg a popurri18.jpg
Tipo de fuente	Escaneos históricos de documentos impresos
Variabilidad visual	Alta: columnas, dobles páginas, anuncios, tipografías decorativas y fondos envejecidos
Uso dentro del proyecto	Detección, grid search, ranking, validación OCR y evaluación con ground truth
Ground truth	Disponible para las 18 imágenes en el archivo ground_truth/ocr_ground_truth.json



(a) popurri01



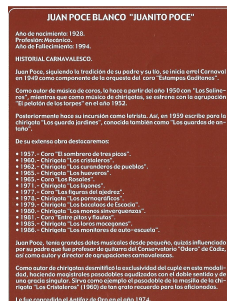
(b) popurri02



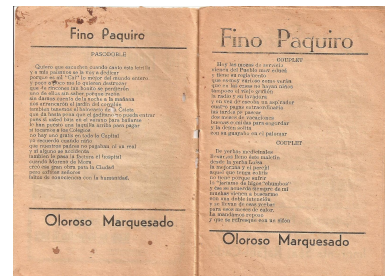
(c) popurri03



(d) popurri04



(e) popurri05



(f) popurri06

Figura 1: Galería del dataset, bloque 1 de 3.



(a) popurri07

(b) popurri08

(c) popurri09



(d) popurri10

(e) popurri11

(f) popurri12

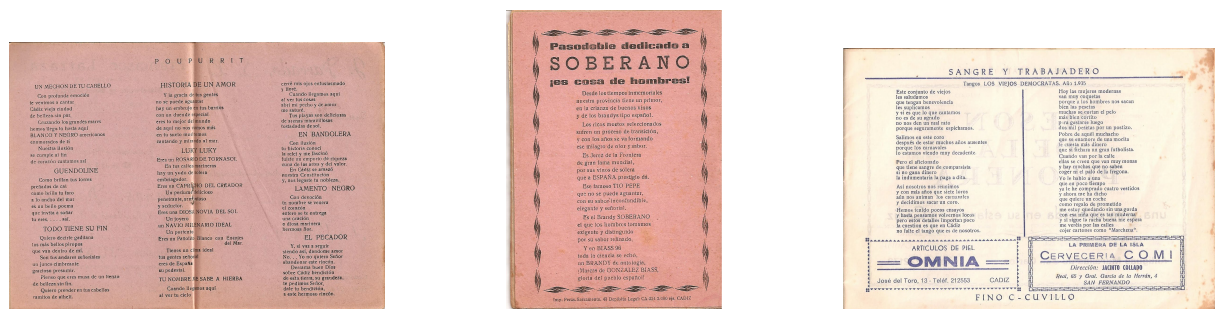
Figura 2: Galería del dataset, bloque 2 de 3.



(a) popurri13

(b) popurri14

(c) popurri15



(d) popurri16

(e) popurri17

(f) popurri18

Figura 3: Galería del dataset, bloque 3 de 3.

Cuadro 2: Inventario del conjunto de imágenes

Archivo	Uso dentro del proyecto
popurri01.jpg	Validación, OCR por regiones y análisis de layout.
popurri02.jpg	Validación, OCR por regiones y análisis de layout.

Archivo	Uso dentro del proyecto
popurri03.jpg	Validación, OCR por regiones y análisis de layout.
popurri04.jpg	Validación, OCR por regiones y análisis de layout.
popurri05.jpg	Validación, OCR por regiones y análisis de layout.
popurri06.jpg	Validación, OCR por regiones y análisis de layout.
popurri07.jpg	Validación, OCR por regiones y análisis de layout.
popurri08.jpg	Validación, OCR por regiones y análisis de layout.
popurri09.jpg	Validación, OCR por regiones y análisis de layout.
popurri10.jpg	Validación, OCR por regiones y análisis de layout.
popurri11.jpg	Validación, OCR por regiones y análisis de layout.
popurri12.jpg	Validación, OCR por regiones y análisis de layout.
popurri13.jpg	Validación, OCR por regiones y análisis de layout.
popurri14.jpg	Validación, OCR por regiones y análisis de layout.
popurri15.jpg	Validación, OCR por regiones y análisis de layout.
popurri16.jpg	Validación, OCR por regiones y análisis de layout.
popurri17.jpg	Validación, OCR por regiones y análisis de layout.
popurri18.jpg	Validación, OCR por regiones y análisis de layout.

5 Proceso de instalación del sistema

El proyecto está preparado principalmente para Windows con Python 3.11, aunque parte del flujo también puede adaptarse a otros entornos. La instalación recomendada se realiza mediante el script `install.bat`, que automatiza el orden correcto de dependencias.

5.1 Requisitos previos

Antes de instalar los paquetes Python es necesario comprobar:

- Python 3.10 o 3.11.
- Tesseract OCR si se quiere utilizar ese motor.
- Git para descargas automáticas de modelos.
- GPU NVIDIA si se pretende usar DeepSeek y acelerar los métodos de layout basados en DL.

El proyecto incluye un comprobador de prerequisites:

```
1 py -3.11 check_prerequisites.py
```

Listing 1: Verificación inicial del entorno

5.2 Instalación recomendada

La instalación recomendada en Windows es:

```
1 install.bat
```

Listing 2: Instalación automática recomendada

Este script realiza las siguientes tareas:

1. Verifica Python 3.11 y Tesseract,
2. Detecta si el equipo tiene GPU o solo CPU,

3. Instala la versión adecuada de PyTorch,
4. Instala PaddleOCR, Docling y el resto de dependencias,
5. Descarga el modelo de DocLayout-YOLO,
6. Copia `sitecustomize.py` a `site-packages` para compatibilidad con DeepSeek.

5.3 Notas de compatibilidad importantes

Una de las partes más delicadas del proyecto ha sido la convivencia entre Docling y DeepSeek-OCR dentro del mismo entorno Python. La configuración validada del proyecto es:

- `transformers==4.53.3`,
- `tokenizers>=0.21.0,<0.22.0`,
- Uso de `sitecustomize.py` para reexponer símbolos esperados por DeepSeek.

También se comprobó que DeepSeek funciona de forma más estable sobre regiones con tamaño 1024 x 1024 que con 640 x 640, donde aparecieron errores internos.

6 Proceso de uso del sistema

El sistema se puede utilizar por fases, desde pruebas individuales hasta evaluación completa.

6.1 Detección de layout

Para detectar regiones de texto en una imagen se utiliza `detect_columns.py`.

```
1 py -3.11 detect_columns.py --image imgs/popurri01.jpg --method opencv --debug
2 py -3.11 detect_columns.py --image imgs/popurri01.jpg --method doclayout --debug
3 py -3.11 detect_columns.py --image imgs/popurri01.jpg --method yolo11 --yolo11-
  size nano --debug
4 py -3.11 detect_columns.py --image imgs/popurri01.jpg --method paddleocr --debug
5 py -3.11 detect_columns.py --image imgs/popurri01.jpg --method docling --debug
```

Listing 3: Ejemplos de detección de layout

6.2 OCR por imagen o por carpeta

Cada motor dispone de scripts específicos de prueba. Un ejemplo sencillo con PaddleOCR es:

```
1 py -3.11 paddle-pruebas.py imgs/popurri01.jpg --method doclayout --debug
2 py -3.11 paddle-pruebas.py imgs/ --method doclayout
```

Listing 4: OCR por columnas sobre una imagen o carpeta

6.3 Experimentación y ranking de layout

El proyecto incluye un grid search sistemático de configuraciones de layout:

```
1 py -3.11 experiment_models.py
2 py -3.11 analyze_experiments.py
```

Listing 5: Grid search y análisis de configuraciones

6.4 Validación OCR multi-motor

La validación más completa se realiza con `validate_ocr.py`, que reutiliza la misma detección por configuración para todos los motores OCR:

```
1 py -3.11 validate_ocr.py --top 3 --ocr-engines easyocr,tesseract,paddle,deepseek
   \
2   --tesseract-cmd "C:\Program Files\Tesseract-OCR\tesseract.exe" \
3   --deepseek-model-path ".\models\DeepSeek-OCR"
```

Listing 6: Validacion multi-motor OCR

6.5 Comparación contra ground truth

La medición final de calidad se obtiene mediante:

```
1 py -3.11 compare_validation_vs_ground_truth.py
```

Listing 7: Comparacion OCR vs ground truth

Las salidas más importantes del proyecto quedan almacenadas en:

- `validation_results/`
- `experiment_results.json`
- `experiment_ranking.csv`
- `ocr_gt_comparison_report.csv`
- `ocr_gt_comparison_report.txt`

7 Desarrollo experimental del proyecto

7.1 Fase 1: detección de layout

La primera fase consistió en validar distintos detectores de layout sobre el conjunto de 18 imágenes. El resultado más llamativo fue que el enfoque heurístico con OpenCV obtuvo una cobertura media muy superior al resto en este dataset concreto.

Esto sugiere que, para este tipo de material histórico, el problema dominante no siempre es la falta de capacidad del modelo profundo, sino la capacidad de adaptarse a estructuras muy regulares y a textos impresos grandes sobre fondos relativamente claros.

7.2 Fase 2: grid search

El grid search recorrió las combinaciones de:

- Familias doclayout y yolo11: `conf x nms_iou x merge_distance`,
- Familias paddleocr y docling: `nms_iou x merge_distance`,
- Método opencv: `merge_distance`.

En total se evaluaron **164 configuraciones** de layout.

7.3 Fase 3: ranking heurístico

El ranking de configuraciones utiliza una fórmula basada en cobertura textual y penalización de solapamientos, cajas vacías, cajas diminutas y duplicados. El mejor resultado global fue OpenCV, con una cobertura media cercana al 99 % y tiempo medio de detección alrededor de 23 ms.

Cuadro 3: Top 10 global del ranking de configuraciones de layout

Rank	Método	conf	NMS	Merge	Cobertura	Tiempo ms	Score
1	opencv	N/A	N/A	15	0.99	23.04	99.30
2	opencv	N/A	N/A	20	0.99	23.53	99.30
3	opencv	N/A	N/A	10	0.99	25.39	99.30
4	opencv	N/A	N/A	5	0.99	24.13	99.30
5	docling	N/A	0.50	20	0.58	44901.34	54.69
6	docling	N/A	0.40	20	0.58	42212.19	54.69
7	docling	N/A	0.30	20	0.58	44027.30	54.69
8	docling	N/A	0.60	20	0.58	58812.06	54.69
9	paddleocr	N/A	0.50	5	0.55	2524.57	53.45
10	paddleocr	N/A	0.40	5	0.55	2897.68	53.45

Cuadro 4: Campos utilizados en las tablas de ranking de layout

Campo	Significado
Rank	Posición de la configuración en el ranking global.
Método	Familia de detector utilizada para localizar regiones de texto.
conf	Umbral de confianza del detector cuando el método lo soporta.
NMS	Umbral de Non-Maximum Suppression aplicado al postprocesado de cajas.
Merge	Distancia en píxeles usada para fusionar cajas cercanas.
Cobertura	Fracción media de texto visible cubierta por las cajas detectadas.
Tiempo ms	Tiempo medio de detección por imagen en milisegundos.
Score	Métrica heurística final del ranking, derivada de cobertura y penalizaciones.

Cuadro 5: Mejor configuración obtenida por cada familia de layout

Método	Configuración	Cobertura	Solap.	Dups	Tiempo ms	Score
OpenCV	mg=15	0.99	0.00	0.00	23.04	99.30
Docling	nms=0.50, mg=20	0.58	0.08	0.00	44901.34	54.69
PaddleOCR	nms=0.50, mg=5	0.55	0.02	0.00	2524.57	53.45
DocLayout	conf=0.10, nms=0.30, mg=5	0.56	0.08	0.06	15881.93	52.72
YOLO11	conf=0.10, nms=0.30, mg=20	0.12	0.01	0.00	—	11.22

Cuadro 6: Campos utilizados en la comparativa por familia de layout

Campo	Significado
Cobertura Solap.	Proporción media de texto recuperado por las cajas detectadas. Solapamiento medio entre cajas; valores altos suelen indicar redundancia.
Dups	Duplicados medios detectados tras el postprocesado.
Tiempo ms	Tiempo medio de detección por imagen.
Score	Puntuación final del ranking heurístico.

La tabla anterior resume una conclusión importante del proyecto: en este dataset concreto, OpenCV supera con mucha claridad a los métodos basados en modelos profundos en cobertura y coste computacional. Docling y PaddleOCR se comportan de forma razonable, mientras que YOLO11 queda claramente por debajo.

7.4 Fase 4: validación OCR multi-motor

Una vez seleccionadas las configuraciones más prometedoras, se ejecutó OCR con varios motores sobre las mismas regiones detectadas. Esto elimina el sesgo que se produciría si cada OCR trabajara sobre segmentaciones distintas.

La comparativa final frente a ground truth mostró que la mejor combinación fue:

OpenCV + DeepSeek-OCR

con una **accuracy de contenido del 95.35 %**.

Cuadro 7: Comparativa de motores OCR sobre la mejor familia de layout (OpenCV)

Motor OCR	CER contenido	WER contenido	Acc. contenido	Reorder ratio	Tiempo ms/img
DeepSeek	0.0383	0.0588	95.35 %	0.1667	65304.02
PaddleOCR	0.0517	0.0662	94.25 %	0.2222	10446.79
Tesseract	0.0745	0.1048	91.34 %	0.2778	1691.77
EasyOCR	0.0970	0.1439	88.43 %	0.5000	17594.06

Esta tabla deja ver el principal compromiso del sistema:

- DeepSeek obtiene la mayor calidad, pero es con diferencia el más costoso en tiempo.
- Tesseract es mucho más rápido y mantiene una calidad bastante competitiva.
- PaddleOCR queda muy equilibrado entre calidad y coste.
- EasyOCR fue el menos preciso de los cuatro en este escenario.

Cuadro 8: Mejor resultado encontrado por familia de layout usando OCR DeepSeek

Layout	CER cont.	WER cont.	Acc. contenido	Disorder	Tiempo ms/img
OpenCV mg10	0.0383	0.0588	95.35 %	0.0353	65304.02
PaddleOCR	0.1614	0.1875	82.82 %	0.2429	116919.61
nms0.30 mg5					
Docling nms0.30 mg20	0.2135	0.2423	77.50 %	0.0986	92417.13

Cuadro 9: Top 20 combinaciones OCR-layout por accuracy de contenido

Layout	OCR	CER cont.	WER cont.	Acc. cont.	Tiempo ms/img
OpenCV mg10	deepseek	0.0383	0.0588	95.35 %	65304.02
OpenCV mg15	deepseek	0.0383	0.0588	95.35 %	67195.96
OpenCV mg20	deepseek	0.0383	0.0588	95.35 %	64783.07
OpenCV mg5	deepseek	0.0383	0.0588	95.35 %	64955.83
OpenCV mg10	paddle	0.0517	0.0662	94.25 %	10446.79
OpenCV mg15	paddle	0.0517	0.0662	94.25 %	10391.83
OpenCV mg20	paddle	0.0517	0.0662	94.25 %	10654.37
OpenCV mg5	paddle	0.0517	0.0662	94.25 %	10588.87
OpenCV mg10	tesseract	0.0745	0.1048	91.34 %	1691.77
OpenCV mg15	tesseract	0.0745	0.1048	91.34 %	1743.20
OpenCV mg20	tesseract	0.0745	0.1048	91.34 %	1700.61
OpenCV mg5	tesseract	0.0745	0.1048	91.34 %	1698.91
OpenCV mg10	easyocr	0.0970	0.1439	88.43 %	17594.06
OpenCV mg15	easyocr	0.0970	0.1439	88.43 %	20446.01
OpenCV mg20	easyocr	0.0970	0.1439	88.43 %	18375.31
OpenCV mg5	easyocr	0.0970	0.1439	88.43 %	18179.83
PaddleOCR nms0.30 mg5	deepseek	0.1614	0.1875	82.82 %	116919.61
PaddleOCR nms0.40 mg5	deepseek	0.1614	0.1875	82.82 %	96239.21
PaddleOCR nms0.30 mg5	tesseract	0.1793	0.2123	80.75 %	6458.41
PaddleOCR nms0.40 mg5	tesseract	0.1793	0.2123	80.75 %	6732.35

Cuadro 10: Tabla complementaria: Top 20 combinaciones OCR-layout por velocidad

Layout	OCR	CER cont.	WER cont.	Acc. cont.	Tiempo ms/img
OpenCV mg10	tesseract	0.0745	0.1048	91.34 %	1691.77
OpenCV mg5	tesseract	0.0745	0.1048	91.34 %	1698.91
OpenCV mg20	tesseract	0.0745	0.1048	91.34 %	1700.61
OpenCV mg15	tesseract	0.0745	0.1048	91.34 %	1743.20
PaddleOCR nms0.30 mg5	tesseract	0.1793	0.2123	80.75 %	6458.41
PaddleOCR nms0.40 mg5	tesseract	0.1793	0.2123	80.75 %	6732.35
Docling nms0.40 mg20	tesseract	0.2130	0.2427	77.51 %	9111.95
Docling nms0.30 mg20	tesseract	0.2130	0.2427	77.51 %	9604.09
Docling nms0.50 mg20	tesseract	0.2130	0.2427	77.51 %	9635.78
Docling nms0.60 mg20	tesseract	0.2235	0.2541	76.43 %	10362.04
OpenCV mg15	paddle	0.0517	0.0662	94.25 %	10391.83
OpenCV mg10	paddle	0.0517	0.0662	94.25 %	10446.79
OpenCV mg5	paddle	0.0517	0.0662	94.25 %	10588.87
OpenCV mg20	paddle	0.0517	0.0662	94.25 %	10654.37
PaddleOCR nms0.30 mg5	easyocr	0.2016	0.2525	77.80 %	15629.34
PaddleOCR nms0.40 mg5	easyocr	0.2016	0.2525	77.80 %	15641.97
OpenCV mg10	easyocr	0.0970	0.1439	88.43 %	17594.06
OpenCV mg5	easyocr	0.0970	0.1439	88.43 %	18179.83
OpenCV mg20	easyocr	0.0970	0.1439	88.43 %	18375.31
OpenCV mg15	easyocr	0.0970	0.1439	88.43 %	20446.01

Cuadro 11: Campos utilizados en las tablas OCR y de comparación con ground truth

Campo	Significado
Layout	Configuración de detección usada antes del OCR.
OCR	Motor de reconocimiento aplicado sobre las regiones detectadas.
CER cont.	Error de caracteres medido sobre contenido, reduciendo el impacto del orden de lectura.
WER cont.	Error de palabras medido sobre contenido.
Acc. cont.	Accuracy agregada derivada de CER y WER de contenido.
Disorder	Medida de desorden respecto al ground truth; cuanto menor, mejor.
Reorder ratio	Proporción de imágenes que necesitarían reordenación posterior.
Tiempo ms/img	Tiempo medio por imagen, sumando detección y OCR.

Esta segunda comparativa muestra que la elección del layout influye tanto como el propio OCR. Incluso usando el mismo motor DeepSeek, la calidad cae de forma notable cuando la detección de regiones es peor o introduce mayor fragmentación del contenido.

7.5 Fase 5 y 6: ground truth y evaluación objetiva

El proyecto incluye `ground_truth/ocr_ground_truth.json`, que permite medir calidad real mediante:

- **CER raw**: Error a nivel de carácter incluyendo orden de lectura.
- **WER raw**: Error a nivel de palabra incluyendo orden.
- **CER content**: Error a nivel de contenido, con menor sensibilidad al orden.
- **WER content**: Equivalente a nivel de palabra.
- **content accuracy**: Medida resumida en escala 0–1.
- **disorder**: Grado de desorden respecto al ground truth.

La disponibilidad de estas métricas es una de las fortalezas principales del trabajo, porque permite distinguir entre un OCR que falla en reconocimiento y un OCR que recupera bien el texto pero con un orden de ensamblado mejorable.

8 Discusión de resultados

Los resultados obtenidos permiten extraer varias conclusiones técnicas relevantes:

1. **El layout domina el rendimiento global.** El mejor OCR no puede compensar del todo una segmentación mediocre.
2. **OpenCV funcionó sorprendentemente bien** en este conjunto concreto, probablemente porque muchas imágenes presentan columnas relativamente bien separadas y estructuras de texto amplias.
3. **DeepSeek-OCR fue el motor más preciso**, pero también el más caro en tiempo.
4. **Tesseract sigue siendo una base muy competitiva** cuando interesa coste bajo y buen rendimiento general.
5. **YOLO11 no se adaptó bien al dataset** y quedó muy lejos del resto en cobertura.

9 Conclusiones

El proyecto ha terminado convertido en una plataforma de evaluación OCR bastante completa, no solo en un conjunto de scripts aislados. Sus aportaciones principales son:

- Un pipeline reproducible desde instalación hasta evaluación final,
- Un conjunto de 18 imágenes históricas con ground truth asociado,
- Una comparativa objetiva de varios métodos de layout,
- Una comparativa objetiva de varios motores OCR,
- Resultados medibles con métricas robustas.

La mejor combinación final encontrada en el proyecto ha sido **OpenCV + DeepSeek-OCR**, que alcanzó una accuracy de contenido del 95.35 %. Sin embargo, si se prioriza velocidad, **OpenCV + Tesseract** ofrece un compromiso muy razonable. Por tanto, la elección óptima depende del objetivo operativo final:

- Máxima calidad: OpenCV + DeepSeek,
- Equilibrio calidad/tiempo: OpenCV + PaddleOCR,
- Mínimo coste temporal: OpenCV + Tesseract.

Como trabajo futuro, los pasos más naturales son mejorar la reordenación de lectura entre regiones, ampliar el ground truth a otros tipos documentales y estudiar estrategias híbridas que usen diferentes layouts según la morfología de la página.

Referencias

- [1] Tesseract OCR. <https://github.com/tesseract-ocr/tesseract>
- [2] PaddleOCR. <https://github.com/PaddlePaddle/PaddleOCR>
- [3] EasyOCR. <https://github.com/JaidedAI/EasyOCR>
- [4] IBM Docling. <https://github.com/docling-project/docling>
- [5] Ultralytics YOLO. <https://github.com/ultralytics/ultralytics>